

Databases (DBMS)

500+ One-Liners for Rapid Revision

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

1. Introduction to DBMS	5
Foundations	5
2. ER Model	5
Entities and Attributes	5
Enhanced ER (EER)	6
ER to Relational Mapping	6
3. Relational Model	6
Concepts and Keys	6
4. Relational Algebra	7
Operators	7
5. Relational Calculus	7
TRC and DRC	7
6. SQL - DDL	7
Data Definition	7
7. SQL - DML	8
Queries and Joins	8
Aggregation and Grouping	8
Subqueries	8
Other Objects	8
8. Normalization	9
Functional Dependencies	9
Normal Forms	9
Decomposition	9
9. File Organization & Indexing	9
File Organization	9
Indexing	10
10. Transactions	10
Concepts and ACID	10
Concurrency Problems	10

11. Serializability	10
Conflict and View	10
12. Concurrency Control Protocols	11
Locking and Deadlock	11
13. Recovery	11
Failures and Logging	11
14. NoSQL Databases	11
Fundamentals	12
CAP, BASE, ACID	12
Sharding and Replication	12
15. Exam Focus & Numeric Recall	12
Quick Numeric Facts	12
Practice Techniques	13
Additional Foundations Nuances	13
Additional ER Nuances	13
Additional Relational Algebra Nuances	14
Additional SQL Nuances	14
Additional Normalization Nuances	14
Additional Indexing Nuances	14
Additional Transaction Nuances	15
Additional NoSQL Nuances	15
16. Transaction Isolation Levels	15
SQL Isolation Levels	15
17. Advanced Concurrency Control	15
Multiple Granularity and MVCC	16
Timestamp Protocol Details	16
18. Advanced Recovery	16
ARIES and Logging	16
19. Distributed Databases	16
Fragmentation and Commit	16
20. Query Processing and Optimization	17
Processing Stages	17

21. Storage and Buffer Management	17
Disks, RAID, and Buffers	17
22. PL/SQL and Programmatic Access	17
Procedural Extensions	17
23. SQL Deep Dive	18
Window Functions and Advanced Clauses	18
Constraints and Keys Extra	18
24. NoSQL Expanded	18
Key-Value Stores	18
Document Stores	19
Column-Family Stores	19
Graph Stores	19
CAP, PACELC, and Consistency	19
Sharding and Replication Extra	19
NoSQL vs SQL Comparison	20
25. More Normalization	20
Higher Normal Forms	20
FD Computation Extra	20
26. More ER and Relational Modeling	20
Additional Modeling Concepts	20
Relational Calculus Extra	21
27. Object and Temporal Concepts	21
Beyond Relational	21
28. Data Warehousing and OLAP	21
Analytical Processing	21
29. More Indexing and Hashing	21
Hash and Tree Structures	21
30. Extra Exam Recall	22
High-Yield Numeric and Naming Facts	22

1. Introduction to DBMS

Foundations

- **Data** are **raw facts**, while **information** is **processed, meaningful data** used for decisions.
- The DIKW value ladder has **4** levels: **Data**, **Information**, **Knowledge**, and **Wisdom**.
- A **database** is a **shared, organized, persistent** collection of logically related data.
- A **DBMS** is the **software** that lets users **define, create, maintain, and control** access to a database.
- The famous **4** relational DBMS examples are **Oracle**, **MySQL**, **PostgreSQL**, and **SQL Server**.
- **Metadata** ("data about data") is stored in the **data dictionary / system catalog**.
- File-processing systems suffer from **8** major drawbacks including **redundancy**, **inconsistency**, **isolation**, and **concurrency anomalies**.
- **Redundancy** is the **cause**, and **inconsistency** is the **effect**, that DBMS controls.
- DBMS provides **controlled redundancy**, not **zero redundancy**, since some duplication aids performance.
- The **4** key advantages of DBMS are **consistency**, **integrity**, **sharing**, and **security**.
- The **ANSI/SPARC** architecture, proposed in **1975**, defines **3** schema levels: **external**, **conceptual**, and **internal**.
- The **external level** (topmost) shows **user views**; the **conceptual** shows the **whole logical schema**; the **internal** shows **physical storage**.
- The **3** data abstraction levels are **physical**, **logical**, and **view**, mapping to internal, conceptual, and external.
- **Logical data independence** changes the **conceptual schema** without altering **external views/applications**.
- **Physical data independence** changes the **internal schema** (storage/indexes) without altering the **conceptual schema**.
- **Physical** data independence is **easier** to achieve than **logical** data independence.
- The **DBA** is responsible for **schema definition**, **authorization (GRANT/REVOKE)**, and **backup/recovery**.
- The **5** user types are **naive**, **application programmers**, **sophisticated**, **specialized**, and **DBA**.
- The **relational model** was proposed by **E.F. Codd** in **1970**, and the **ER model** by **Peter Chen** in **1976**.
- The **4** classic data models are **hierarchical (tree)**, **network (graph)**, **relational (tables)**, and **ER**.
- The **hierarchical model** supports only **1:N** (tree), while the **network model** supports **M:N** (graph).
- A DBMS has **2** main engines: the **query processor** and the **storage manager**.
- The **query optimizer** chooses the **lowest-cost** execution plan and is part of the **query processor**.
- The **buffer manager** brings **disk blocks** into **main memory** and is central to performance.

2. ER Model

Entities and Attributes

- An **entity** is a **single distinguishable object**; an **entity set** is a **collection** of same-type entities.
- An entity set is drawn as a **rectangle**, and an attribute as an **ellipse**.
- The **5** attribute classifications are **simple/composite**, **single/multivalued**, **stored/derived**, **key**, and **NULL**.
- A **composite** attribute (e.g., **Name**) can be divided; a **simple/atomic** one (e.g., **Age**) cannot.
- A **multivalued** attribute is drawn as a **double ellipse** (e.g., **PhoneNumbers**).
- A **derived** attribute is drawn as a **dashed ellipse** (e.g., **Age** from **DateOfBirth**).
- A **key** attribute is shown with an **underline** in the ER diagram.
- A **relationship set** is drawn as a **diamond** and associates **2 or more** entities.
- The **degree** of a relationship is the **number of participating entity sets**: **unary (1)**, **binary (2)**, **ternary (3)**.

- A **recursive/self** relationship has degree **1 (unary)** even though it visually connects two ends.
- The **3** mapping cardinalities are **1:1**, **1:N**, and **M:N**.
- Cardinality ratio is set by the **maximum** association count, while **participation** is set by the **minimum**.
- **Total participation** (drawn as a **double line**) means **every** entity must participate.
- **Partial participation** (single line) means **some** entities may not participate.
- The **4** key types in ER are **super**, **candidate**, **primary**, and **alternate** keys.
- A **candidate key** is a **minimal super key**; a **primary key** is the chosen candidate key (**unique + NOT NULL**).
- A **weak entity** has **no key of its own**, is drawn as a **double rectangle**, with a **double-diamond** identifying relationship.
- A weak entity's full identifier = **owner's primary key + its partial key (discriminator)**.
- A **partial key** is shown by a **dashed underline** and a weak entity always has **total participation**.
- A **strong entity** has its **own primary key** and exists **independently**.

Enhanced ER (EER)

- **Specialization** is **top-down (split)**; **generalization** is **bottom-up (combine)**.
- The **ISA** triangle connects a **subclass** to its **superclass** in EER diagrams.
- Subclasses **inherit** all **attributes and relationships** of the superclass.
- The **disjoint (d)** constraint allows an entity in **at most one** subclass; **overlapping (o)** allows **many**.
- **Total** specialization requires every superclass entity to be in **some** subclass; **partial** does not.
- **Aggregation** treats a **relationship as a higher-level entity**, enabling relationships among relationships.

ER to Relational Mapping

- A **1:N** relationship is mapped by placing the **1-side PK as a foreign key** on the **N-side** table (no new table).
- An **M:N** relationship **requires a separate junction table** with PK = **both entity PKs**.
- A **1:1** relationship places the **FK on either side**, preferably the side with **total participation**.
- A **multivalued** attribute requires a **separate table**; a **composite** attribute becomes **multiple columns**.
- Mapping a weak entity gives a table whose PK = **owner PK + partial key**, with owner PK as a **foreign key**.
- The **single-table** generalization mapping causes many **NULL** values for inapplicable subclass attributes.

3. Relational Model

Concepts and Keys

- The **6** relational terms are **relation (table)**, **tuple (row)**, **attribute (column)**, **domain**, **degree**, and **cardinality**.
- **Degree** = number of **columns (attributes)**; **cardinality** = number of **rows (tuples)**.
- A **domain** is the set of **atomic allowed values** for an attribute.
- A **relation schema** is the **fixed structure (intension)**; a **relation instance** is the **current data (extension)**.
- The **4** properties of relations are **no duplicate tuples**, **unordered tuples**, **unordered attributes**, and **atomic values**.
- Pure relational model has **no duplicate rows**, but **SQL tables** are **multisets (allow duplicates)**.
- **NULL** represents **unknown, inapplicable, or missing** data and is **not equal to 0 or empty string**.
- SQL uses **three-valued logic** with values **TRUE**, **FALSE**, and **UNKNOWN**.
- Any comparison with **NULL** yields **UNKNOWN**; use **IS NULL** / **IS NOT NULL** instead.
- **WHERE** keeps a row only when the predicate is **TRUE**, never when **UNKNOWN**.
- The **5** relational key types are **super**, **candidate**, **primary**, **foreign**, and **composite**.
- A **foreign key can be NULL**, but a **primary key can never be NULL** (entity integrity).

- The **4** integrity constraints are **domain**, **key**, **entity integrity**, and **referential integrity**.
- **Entity integrity** means **PK is not NULL**; **referential integrity** means **FK matches a PK or is NULL**.
- The **4** referential actions are **CASCADE**, **SET NULL**, **SET DEFAULT**, and **RESTRICT/NO ACTION**.
- **ON DELETE CASCADE** deletes child rows automatically; **RESTRICT** blocks the parent delete.

4. Relational Algebra

Operators

- Relational algebra is **procedural** (specifies **how**) and has **6** basic operators.
- The **6** primitive operators are **selection**, **projection**, **union**, **set difference**, **Cartesian product**, and **rename**.
- **Selection (sigma)** filters **rows (horizontal)**; **projection (pi)** selects **columns (vertical)**.
- **Projection in relational algebra removes duplicates**, unlike SQL SELECT which keeps them.
- **Rename (rho)** renames relations/attributes and is essential for **self-joins**.
- The **3** set operations **union**, **intersection**, **difference** require **union-compatibility** (same degree + domains).
- **Set difference** is **NOT commutative**: **R - S** differs from **S - R**.
- **Cartesian product** of m and n rows gives **m*n rows** and **a+b columns**.
- A **natural join** with **no common attribute** degenerates into a **Cartesian product**.
- The **4** outer-join behaviors: **inner (none kept)**, **left**, **right**, and **full** preserve unmatched tuples with **NULL**.
- **Equijoin** keeps **both** join columns; **natural join** keeps **one**.
- The **division (÷)** operator answers "**for all**" / **universal** queries.
- **Codd's theorem** states **relational algebra**, **safe TRC**, and **safe DRC** have **equal expressive power**.

5. Relational Calculus

TRC and DRC

- Relational calculus is **non-procedural (declarative)**, based on **first-order predicate logic**.
- **TRC** variables range over **tuples** in form **t | P(t)**; **DRC** variables range over **domain values**.
- The **2** quantifiers in calculus are **existential (∃)** and **universal (∀)**.
- **QBE (Query By Example)** is based on **Domain Relational Calculus (DRC)**.
- A **safe expression** guarantees a **finite** result drawn from the **active domain**.
- Relational algebra is **procedural** but has the **same power** as the **declarative** calculus.

6. SQL - DDL

Data Definition

- **SQL** originated as **SEQUEL** at **IBM** and is a **declarative** language.
- SQL has **5** sub-languages: **DDL**, **DML**, **DCL**, **TCL**, and **DQL**.
- **DDL** commands include **CREATE**, **ALTER**, **DROP**, and **TRUNCATE**.
- **CHAR(n)** is **fixed-length and space-padded**; **VARCHAR(n)** stores only **actual characters**.
- The **6** common constraints are **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL**, **CHECK**, and **DEFAULT**.
- A **UNIQUE** column **allows one NULL**, while a **PRIMARY KEY** **allows none**.
- **DELETE** is **DML** (rollback-able, has WHERE), **TRUNCATE** is **DDL** (no WHERE, auto-commit, faster).

- **DROP TABLE** removes both **structure and data**, while **TRUNCATE** removes **only rows**.
- **CREATE INDEX** speeds **SELECT/WHERE/JOIN** but **slows INSERT/UPDATE/DELETE**.

7. SQL - DML

Queries and Joins

- **SELECT** keeps **duplicates**; only **DISTINCT** removes them.
- The WHERE clause uses operators **BETWEEN**, **IN**, **LIKE**, **IS NULL**, and **logical AND/OR/NOT**.
- **BETWEEN a AND b** is **inclusive** of both endpoints.
- In **LIKE**, **%** matches **any string** and **_** matches **exactly one character**.
- Logical operator precedence is **NOT > AND > OR**.
- **ORDER BY** defaults to **ASC** and is the **last** clause executed.
- **UPDATE/DELETE without WHERE** affect **every row** in the table.
- The **6** SQL join types are **INNER**, **LEFT**, **RIGHT**, **FULL OUTER**, **CROSS**, and **SELF**.
- A **CROSS JOIN** of m and n rows produces **m*n** rows.
- A **LEFT JOIN** result always has **>=** the left table's row count.
- The **4** SQL set operators are **UNION**, **UNION ALL**, **INTERSECT**, and **EXCEPT/MINUS**.
- **UNION** removes duplicates (extra sort), while **UNION ALL** keeps them and is **faster**.
- **MINUS** (Oracle) is equivalent to **EXCEPT** (SQL Server/PostgreSQL).

Aggregation and Grouping

- The **5** aggregate functions are **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**.
- All aggregates except **COUNT(*)** ignore **NULLs**.
- **COUNT(*)** counts **all rows**, while **COUNT(col)** counts only **non-NULL** values.
- **AVG(col)** divides by the count of **non-NULL** values, not total rows.
- **WHERE** filters **rows before grouping**; **HAVING** filters **groups after aggregation**.
- Aggregates **cannot** be used in **WHERE**; they must go in **HAVING**.
- The logical clause evaluation order is **FROM**, **WHERE**, **GROUP BY**, **HAVING**, **SELECT**, **DISTINCT**, **ORDER BY**.
- **SELECT** runs **after** WHERE, so **aliases** usually can't be used in WHERE but can in ORDER BY.

Subqueries

- A subquery in **FROM** is a **derived table**; in **SELECT** it must be **scalar**.
- **EXISTS** returns true if the subquery yields **at least one row**; **NOT EXISTS** models **"for all"**.
- **> ANY** means greater than the **minimum**; **> ALL** means greater than the **maximum**.
- **= ANY** equals **IN**, and **<> ALL** equals **NOT IN**.
- A **correlated subquery** references the **outer query** and is re-evaluated **once per outer row**.
- **NOT IN** with a **NULL** in its list returns **no rows**; use **NOT EXISTS** to be safe.

Other Objects

- A **view** is a **virtual table** storing **no data**, used for **security** and **logical data independence**.
- A **materialized view** stores results **physically** and must be **refreshed**.
- Views with **aggregation**, **GROUP BY**, or **DISTINCT** are **not updatable**.
- A **trigger** fires **automatically** on **INSERT/UPDATE/DELETE**, with timing **BEFORE** or **AFTER**.

- **Stored procedures** are **precompiled** routines that reduce **network traffic**.
- **GRANT/REVOKE** are **DCL** commands; **WITH GRANT OPTION** lets a grantee **re-grant** privileges.

8. Normalization

Functional Dependencies

- An FD $X \rightarrow Y$ means tuples agreeing on **X** must agree on **Y**; **X** is the **determinant**.
- An FD is **trivial** iff the **right side is a subset of the left side**.
- **Armstrong's axioms** have **3** primary rules: **reflexivity**, **augmentation**, and **transitivity**.
- The **3** derived rules are **union**, **decomposition**, and **pseudotransitivity**.
- **Attribute closure X^+** is the set of all attributes determined by **X**.
- **X** is a **super key** iff **$X^+ = \text{all attributes (R)}$** .
- An attribute appearing **only on the left** of FDs is in **every candidate key**.
- An attribute appearing **only on the right** is in **no candidate key**.
- A **prime attribute** belongs to **some candidate key**; a **non-prime** belongs to **none**.
- A **minimal/canonical cover** has **single-attribute RHS**, **no redundant FD**, and **no extraneous LHS attribute**.

Normal Forms

- The **3** anomalies are **insertion**, **deletion**, and **update**, all caused by **redundancy**.
- **1NF** requires all values to be **atomic (single-valued)**.
- **2NF** requires **no partial dependency** of a non-prime attribute on part of a key.
- **Partial dependency** only arises with a **composite** key, so single-attribute keys imply **2NF**.
- **3NF** requires **no transitive dependency**: for $X \rightarrow A$, either **X is super key** or **A is prime**.
- **BCNF** requires that for **every** non-trivial FD $X \rightarrow A$, **X is a super key**.
- **BCNF is stricter** than **3NF**; every BCNF relation is in 3NF but not vice versa.
- The normal-form order is **$1NF < 2NF < 3NF < BCNF < 4NF < 5NF$** .
- **4NF** handles **multivalued dependencies (MVDs)**; **5NF** handles **join dependencies**.
- The **highest normal form** of a relation is the **strictest one** all its FDs satisfy.

Decomposition

- A 2-way decomposition is **lossless** iff **$R1 \cap R2$ is a super key of $R1$ or $R2$** .
- A **lossy** decomposition produces **spurious (extra) tuples** on rejoin.
- A **dependency-preserving** decomposition lets constraints be checked **without joins**.
- **3NF** decomposition can always be both **lossless and dependency-preserving**.
- **BCNF** decomposition is always **lossless** but **may not preserve dependencies**.

9. File Organization & Indexing

File Organization

- **Fixed-length records** allow **$O(1)$ offset addressing** but **waste space**.
- **Heap** organization gives **$O(1)$ insert** (append) but **$O(n)$ search** (full scan).
- **Sequential/ordered** files allow **$O(\log n)$ binary search** and good **range queries**.
- **Hash** file organization gives **$O(1)$ average equality search** but is **poor for range** queries.

- **Blocking factor $bfr = \text{floor}(B/R)$** = records per block; block count **$b = \text{ceil}(r/bfr)$** .

Indexing

- An **index** is a (**search-key, pointer**) structure that speeds up retrieval.
- There can be only **one clustering (primary) index** per table but **many secondary** indexes.
- A **primary/clustering** index is on the **ordering field**; a **secondary** index is always **dense**.
- A **dense** index has an entry for **every record**; a **sparse** index has **one entry per block**.
- A **sparse** index requires the data to be **ordered** on the index field.
- A **multilevel** index builds an **index on the index**, reducing search to **$O(\log_{\text{fanout}} n)$** .
- A **B-tree** of order **m** has at most **m children** and **$m-1$ keys** per node, with all leaves at the **same level**.
- In a **B-tree**, **data pointers exist in all nodes** (internal and leaf).
- In a **B+ tree**, **all data pointers are in the leaves** and **leaves are linked** in a list.
- **B+ tree** leaf linkage makes **range queries** efficient; it has **higher fan-out** than a B-tree.
- A B+ tree of order p has up to **p pointers** and **$p-1$ keys** per internal node.
- B+ tree height is **$\text{ceil}(\log_p N)$** , and higher **fan-out** means **fewer disk I/Os**.
- **Static hashing** suffers **overflow** on growth; **extendible hashing** uses a **directory + global/local depth**.
- Most real DBMS indexes are **B+ trees**, not B-trees.

10. Transactions

Concepts and ACID

- A **transaction** is a **logical all-or-nothing unit of work**.
- The **2** basic transaction operations are **read(X)** and **write(X)**.
- The **4** ACID properties are **Atomicity**, **Consistency**, **Isolation**, and **Durability**.
- **Atomicity** (all-or-nothing) is ensured by the **transaction/recovery manager**.
- **Isolation** is ensured by the **concurrency-control manager**.
- **Durability** (survives crashes) is ensured by the **recovery manager via logs**.
- A transaction passes through **5** states: **active**, **partially committed**, **committed**, **failed**, and **aborted/terminated**.
- On success, the path is **Active -> Partially committed -> Committed**.
- A **serial schedule** has **no interleaving** and is always **consistent**.
- A **concurrent schedule** interleaves operations to improve **throughput and utilization**.

Concurrency Problems

- The **4** concurrency problems are **lost update**, **dirty read**, **unrepeatable read**, and **phantom read**.
- A **lost update** is a **W-W** problem where one write **overwrites** another.
- A **dirty read** is a **W-R** problem: reading **uncommitted** data that may be **aborted**.
- An **unrepeatable read** is an **R-W** problem: re-reading a row gives a **different value**.
- A **phantom read** occurs when a range re-query returns **new or removed rows**.

11. Serializability

Conflict and View

- Two operations **conflict** if they are from **different transactions**, access the **same item**, and **≥ 1 is a write**.
- The **3** conflict types are **W-W**, **R-W**, and **W-R**; **R-R never conflicts**.
- A schedule is **conflict serializable** iff its **precedence graph is acyclic**.
- A **cycle** in the precedence graph means the schedule is **not conflict serializable**.
- A **topological sort** of an acyclic precedence graph gives an **equivalent serial order**.
- **Conflict serializability** is a **subset** of **view serializability**.
- **View serializability** permits **blind writes** and its test is **NP-complete**.
- The recoverability hierarchy is **Strict $\subset$ Cascadeless $\subset$ Recoverable**.
- A **recoverable** schedule commits only **after** transactions it **read from** commit.
- A **cascadeless** schedule reads only **committed** data, avoiding **cascading rollback**.
- A **strict** schedule allows no **read or write** of an item until the writer **commits/aborts**.

12. Concurrency Control Protocols

Locking and Deadlock

- The **2** lock modes are **shared (S)** for reading and **exclusive (X)** for writing.
- Multiple transactions may hold **S** on an item, but only **one** may hold **X**.
- **2PL (Two-Phase Locking)** guarantees **conflict serializability** via **growing** and **shrinking** phases.
- In **2PL**, the **growing** phase only **acquires** locks and the **shrinking** phase only **releases** them.
- **Basic 2PL** does **not** prevent **deadlocks** or **cascading rollbacks**.
- **Strict 2PL** holds all **exclusive locks until commit**, making it **cascadeless**.
- **Rigorous 2PL** holds **all locks (S and X) until commit**.
- The **4** Coffman deadlock conditions are **mutual exclusion**, **hold and wait**, **no preemption**, and **circular wait**.
- **Wait-die** is **non-preemptive**: the **older** transaction **waits**, the **younger dies**.
- **Wound-wait** is **preemptive**: the **older** transaction **wounds (aborts)** the younger, the **younger waits**.
- In both deadlock schemes, the **older** transaction is **favored (never killed)**.
- **Timestamp ordering** assigns each transaction a **timestamp** and is **deadlock-free** but may cause **starvation**.
- The **Thomas write rule ignores** an **obsolete write** instead of aborting.
- **Optimistic (validation-based) CC** has **3** phases: **Read**, **Validation**, and **Write**.
- Optimistic CC suits **read-heavy, low-conflict** workloads.

13. Recovery

Failures and Logging

- The **3** failure types are **transaction failure**, **system crash**, and **disk failure**.
- A **log** record has the form **<Ti, X, old_value, new_value>**.
- **Deferred update** applies writes **only after commit** and needs **REDO only**.
- **Immediate update** applies writes **before commit** and needs **UNDO + REDO**.
- A **committed** transaction is **REDONE**; an **uncommitted** one is **UNDONE**.
- **Write-Ahead Logging (WAL)** requires the **log** to reach stable storage **before** the data.
- A **checkpoint** bounds the **recovery scan**, reducing recovery time.
- **Shadow paging** uses **shadow + current page tables** and needs **no log**, but causes **fragmentation**.

14. NoSQL Databases

Fundamentals

- **NoSQL** stands for "**Not Only SQL**" and denotes **non-relational, horizontally scalable** databases.
- The **4** NoSQL categories are **key-value**, **document**, **column-family**, and **graph** stores.
- **Key-value** stores access data by **key only** in **O(1)** and treat the value as an **opaque blob**.
- Key-value examples include **Redis** (port **6379**), **DynamoDB**, and **Memcached**.
- **Document** stores hold **JSON/BSON** documents and allow **querying on fields**.
- **MongoDB** stores **BSON** on port **27017** and allows **schema-flexible** documents per collection.
- **Column-family** stores (**Cassandra**, **HBase**) descend from **Google Bigtable** and store data by **column family**.
- **Cassandra** is **masterless (peer-to-peer)**, uses **consistent hashing**, and serves CQL on port **9042**.
- **Graph** databases model **nodes, edges, and properties** with relationships as **first-class citizens**.
- **Neo4j** is a graph database using the **Cypher** query language.

CAP, BASE, ACID

- The **CAP theorem** (Brewer, **2000**) says a distributed system can guarantee at most **2** of **Consistency**, **Availability**, **Partition tolerance**.
- **Consistency** means every read sees the **latest write**; **Availability** means every request gets a **response**.
- **Partition tolerance** means the system survives **network partitions**, which are **unavoidable** when distributed.
- Real distributed systems choose between **CP** and **AP** since **P is mandatory**.
- **CP** examples include **HBase** and **MongoDB**; **AP** examples include **Cassandra** and **DynamoDB**.
- A **CA** system is effectively **single-node** (cannot tolerate partitions).
- **BASE** stands for **Basically Available**, **Soft state**, and **Eventual consistency**.
- **Eventual consistency** means replicas **converge eventually**, not instantly.
- **ACID** favors **strong consistency**, while **BASE** favors **availability and scale**.

Sharding and Replication

- **Sharding** is **horizontal partitioning** that splits **rows** across nodes (shards).
- A **shard key** decides which **shard** a record belongs to.
- The **3** sharding strategies are **range-based**, **hash-based**, and **directory/consistent-hashing**.
- **Hash-based** sharding spreads load evenly but **breaks range queries**.
- **Consistent hashing** minimizes **data movement** when nodes are added/removed.
- **Replication** keeps **data copies** on multiple nodes for **availability and read scaling**.
- The **3** replication models are **master-slave**, **master-master**, and **peer-to-peer**.
- **Synchronous** replication is **consistent but slow**; **asynchronous** is **fast but risks data loss**.
- A **quorum** system gives strong consistency when **$R + W > N$** .
- **Sharding** scales **capacity**, while **replication** improves **availability/fault tolerance**.

15. Exam Focus & Numeric Recall

Quick Numeric Facts

- There are **3** ANSI/SPARC schema levels: **external**, **conceptual**, and **internal**.
- There are **3** data abstraction levels: **physical**, **logical**, and **view**.
- There are **2** types of data independence: **logical** and **physical**.
- There are **4** classic data models: **hierarchical**, **network**, **relational**, and **ER**.

- There are **3** relationship degrees: **unary**, **binary**, and **ternary**.
- There are **3** mapping cardinalities: **1:1**, **1:N**, and **M:N**.
- There are **4** ER key types: **super**, **candidate**, **primary**, and **alternate**.
- There are **6** basic relational-algebra operators: σ , π , $\cup$, $-$, $\times$, and ρ .
- There are **4** integrity constraints: **domain**, **key**, **entity**, and **referential**.
- There are **4** referential actions: **CASCADE**, **SET NULL**, **SET DEFAULT**, and **RESTRICT**.
- There are **5** SQL sub-languages: **DDL**, **DML**, **DCL**, **TCL**, and **DQL**.
- There are **5** aggregate functions: **COUNT**, **SUM**, **AVG**, **MIN**, and **MAX**.
- There are **6** SQL join types: **INNER**, **LEFT**, **RIGHT**, **FULL**, **CROSS**, and **SELF**.
- There are **4** SQL set operators: **UNION**, **UNION ALL**, **INTERSECT**, and **EXCEPT/MINUS**.
- There are **3** primary Armstrong axioms: **reflexivity**, **augmentation**, and **transitivity**.
- There are **3** anomalies: **insertion**, **deletion**, and **update**.
- There are **6** normal forms in order: **1NF**, **2NF**, **3NF**, **BCNF**, **4NF**, **5NF**.
- There are **4** ACID properties: **Atomicity**, **Consistency**, **Isolation**, and **Durability**.
- There are **5** transaction states: **active**, **partially committed**, **committed**, **failed**, **aborted**.
- There are **4** concurrency problems: **lost update**, **dirty read**, **unrepeatable read**, and **phantom**.
- There are **3** conflict types: **W-W**, **R-W**, and **W-R**.
- There are **4** Coffman deadlock conditions: **mutual exclusion**, **hold and wait**, **no preemption**, and **circular wait**.
- There are **2** lock modes: **shared (S)** and **exclusive (X)**.
- There are **3** optimistic CC phases: **Read**, **Validation**, and **Write**.
- There are **3** failure types: **transaction**, **system**, and **disk**.
- There are **4** NoSQL categories: **key-value**, **document**, **column-family**, and **graph**.
- The **CAP theorem** lets you choose at most **2** of **3** properties.
- **Redis** runs on port **6379**, **MongoDB** on **27017**, and **Cassandra CQL** on **9042**.

Practice Techniques

- To find candidate keys, compute the **closure** of **left-only** attributes and extend until it equals **R**.
- To test losslessness of **R1**, **R2**, check whether **$R1 \cap R2$** is a **super key** of either.
- To test conflict serializability, check the **precedence graph** for a **cycle**.
- To map relational algebra to SQL: $\sigma \rightarrow$ **WHERE**, $\pi \rightarrow$ **SELECT**, $\bowtie \rightarrow$ **JOIN**, $\times \rightarrow$ **CROSS JOIN**, $\div \rightarrow$ **NOT EXISTS**.
- A **Cartesian product** of m and n rows yields **$m \times n$** rows; **UNION** yields **$\leq m+n$** distinct rows.
- An **INTERSECT** yields **$\leq \min(m,n)$** rows.

Additional Foundations Nuances

- A **data model** provides tools to describe **data**, **relationships**, **semantics**, and **constraints**.
- The **external/conceptual** and **conceptual/internal** are the **2** mappings in the 3-schema architecture.
- **IBM IMS** is the classic **hierarchical** DBMS, and **CODASYL/DBTG** defined the **network** model.
- Adding an **index** tests **physical** data independence; adding a **column** tests **logical** data independence.
- The **data dictionary** stores the **schema/metadata**, not user data.
- The **query processor** has **3** parts: **DDL interpreter**, **DML compiler**, and **evaluation engine**.
- The **storage manager** includes the **authorization**, **transaction**, **file**, and **buffer** managers.

Additional ER Nuances

- **Composite** attributes can nest, but only their **leaf simple components** are stored as columns.
- A **double ellipse** = **multivalued**; a **dashed ellipse** = **derived**; an **underline** = **key**.
- A **ternary** relationship has degree **3** and cannot always be replaced by **binary** ones.
- An **identifying relationship** (double diamond) connects a **weak entity** to its **owner**.
- A weak entity always shows **total participation** in its identifying relationship.
- **Generalization** reduces **redundancy** by extracting **common attributes** into a superclass.

Additional Relational Algebra Nuances

- **Selection** is **commutative**: the order of stacked σ conditions does not matter.
- **Cascade of selections** equals a single selection with **AND** of the conditions.
- **Cascade of projections** keeps only the **outermost** attribute list.
- **Natural join** is **associative** and **commutative** (ignoring column order).
- A **theta join** equals $\sigma_{\theta}(R \times S)$, combining **Cartesian product** and **selection**.
- **Outer joins** pad missing values with **NULL** to preserve unmatched tuples.

Additional SQL Nuances

- **DISTINCT** applies to the **entire row** (all selected columns together).
- **GROUP BY** on multiple columns groups by each **unique combination**.
- A **HAVING** without **GROUP BY** treats the whole table as **one group**.
- **ORDER BY** can reference a **column position number** or a **SELECT alias**.
- **CASE** expressions provide **conditional logic** inside SELECT.
- **COALESCE** returns the **first non-NULL** argument.
- **NULLIF(a,b)** returns **NULL** when **a = b**, else **a**.
- A **self join** joins a table to itself using **table aliases**.
- **INNER JOIN** can be written as a **comma join with a WHERE** condition.
- **TCL** commands are **COMMIT**, **ROLLBACK**, and **SAVEPOINT**.

Additional Normalization Nuances

- The **closure** algorithm runs until **no new attributes** can be added (a fixed point).
- **Two FD sets** are equivalent iff each **covers** the other (**F+ = G+**).
- A relation with **no non-trivial FDs except super keys** is automatically in **BCNF**.
- **2NF** violations involve a **non-prime attribute** dependent on **part of a candidate key**.
- **3NF** violations involve a **non-prime attribute** dependent on **another non-prime attribute**.
- **BCNF** violations occur when a **prime attribute** depends on a **non-super-key**.
- **Lossless + dependency preserving** is always possible up to **3NF**.

Additional Indexing Nuances

- A **search key** (index field) is **different** from a **candidate key**.
- **Clustering index** entries match the **physical order** of records, enabling **sparse** indexing.
- A **secondary index** on a **non-key** field may use a **bucket of pointers**.
- **B+ tree internal nodes** store **keys only** for routing, not data pointers.
- A **B+ tree** of order p has between **ceil(p/2)** and **p** children per internal node (except root).
- **Disk I/O** is minimized by maximizing the **fan-out** (keys per node).
- **Extendible hashing** doubles the **directory** when a bucket overflows and **global depth** increases.

Additional Transaction Nuances

- **Consistency** in ACID is partly the **application's** responsibility, unlike the other 3.
- A **schedule** preserves the **relative order** of operations within each transaction.
- **Serializability** is the standard **correctness criterion** for concurrent schedules.
- **Cascading rollback** wastes work because one **abort** forces **dependent** aborts.
- **Strict 2PL** is the most widely used protocol in **commercial DBMS**.
- **Deadlock** requires all **4** Coffman conditions to hold **simultaneously**.
- **Deadlock detection** uses a **wait-for graph**; a **cycle** indicates deadlock.
- **Starvation** differs from **deadlock**: a transaction is **repeatedly delayed** but not blocked in a cycle.

Additional NoSQL Nuances

- **NoSQL** typically scales **horizontally (scale-out)**, while RDBMS scales **vertically (scale-up)**.
- **Document** stores are like **key-value** stores where the **value is queryable**.
- **Wide-column** stores write **by column family**, ideal for **time-series** and **write-heavy** loads.
- **Graph** databases beat relational **joins** for **deeply connected** traversal queries.
- **Eventual consistency** is a **BASE** property aligned with **AP** systems.
- **Strong consistency** aligns with **CP** systems and **ACID**.
- A **hot spot** is a sharding risk where one shard receives **disproportionate** load.
- **Replication lag** causes **stale reads** in **master-slave** setups.
- **Master-master** replication risks **write conflicts** needing **resolution**.
- **Quorum reads/writes** ($R + W > N$) trade **latency** for **consistency**.
- **Memcached** and **Redis** are popular **key-value** caching stores.
- **CouchDB** is a **document** store that is **AP** and uses **MVCC**.
- **Amazon DynamoDB** is an **AP**, **key-value/document** managed store using **consistent hashing**.
- Choosing **SQL vs NoSQL** depends on **consistency/join needs** versus **scale/flexibility** needs.

16. Transaction Isolation Levels

SQL Isolation Levels

- The SQL standard defines **4** isolation levels: **Read Uncommitted**, **Read Committed**, **Repeatable Read**, and **Serializable**.
- **Read Uncommitted** is the **lowest** isolation level and permits **dirty reads**, **non-repeatable reads**, and **phantoms**.
- **Read Committed** prevents **dirty reads** but still allows **non-repeatable reads** and **phantoms**.
- **Repeatable Read** prevents **dirty** and **non-repeatable reads** but still allows **phantom reads**.
- **Serializable** is the **highest** level and prevents all **3** anomalies: **dirty**, **non-repeatable**, and **phantom** reads.
- The default isolation level in most systems is **Read Committed** (e.g., **Oracle** and **PostgreSQL**).
- **MySQL InnoDB** uses **Repeatable Read** as its **default** isolation level.
- Higher isolation gives **more consistency** but **less concurrency** (a direct trade-off).
- **Snapshot isolation** reads from a **committed snapshot** and avoids **read locks** using **MVCC**.
- **Write skew** is an anomaly that **snapshot isolation** can still allow despite being near-serializable.

17. Advanced Concurrency Control

Multiple Granularity and MVCC

- **Multiple granularity locking** organizes data as a **tree**: **database** -> **file/area** -> **page** -> **record**.
- **Intention locks** come in **3** types: **IS** (intention shared), **IX** (intention exclusive), and **SIX** (shared + intention exclusive).
- An **intention lock** on a node signals that an **explicit lock** exists at a **lower (finer)** level.
- **MVCC (Multi-Version Concurrency Control)** keeps **multiple versions** so **readers never block writers**.
- **MVCC** is used by **PostgreSQL**, **Oracle**, and **MySQL InnoDB** to boost **read concurrency**.
- In **MVCC**, each transaction sees a **consistent snapshot** as of its **start timestamp**.
- The **lock compatibility matrix** shows **S-S compatible** but **S-X, X-S, and X-X all incompatible**.
- **Lock conversion** upgrades **S to X** (in growing phase) or downgrades **X to S** (in shrinking phase).
- **Graph-based (tree) protocol** ensures serializability and is **deadlock-free** but requires **advance knowledge** of access order.
- **Lock granularity** trades off **concurrency (fine)** against **overhead (coarse)**.

Timestamp Protocol Details

- Timestamp ordering keeps **2** stamps per item: **read-timestamp (RTS)** and **write-timestamp (WTS)**.
- A read by **Ti** is rejected if **TS(Ti) < WTS(X)**, causing **Ti** to be **rolled back**.
- **Strict timestamp ordering** delays reads/writes until the writer **commits**, ensuring **strict + deadlock-free** schedules.
- **Multiversion timestamp ordering** never rejects a **read**, always returning an **appropriate version**.

18. Advanced Recovery

ARIES and Logging

- **ARIES** recovery has **3** phases in order: **Analysis**, **Redo**, and **Undo**.
- **ARIES** uses an **LSN (Log Sequence Number)** to uniquely identify each **log record**.
- **ARIES** maintains **2** key in-memory tables: the **Transaction Table** and the **Dirty Page Table**.
- **ARIES** follows the **"repeating history"** principle by redoing **all** actions before undoing **losers**.
- A **CLR (Compensation Log Record)** is written during **undo** so undo work is **not repeated** after a second crash.
- The **3** log record types beyond updates are **<start>**, **<commit>**, and **<abort>** records.
- **Fuzzy checkpointing** allows transactions to **continue** while the checkpoint is being written.
- **UNDO** restores the **old value**; **REDO** applies the **new value** from the log.
- **Force** policy writes updated pages at **commit** (no redo needed); **no-force** requires **redo**.
- **Steal** policy allows writing **uncommitted** pages to disk and therefore requires **undo**.
- Most DBMS use a **steal/no-force** buffer policy needing both **UNDO and REDO**.
- A **checkpoint** record lists **active transactions** at the time it is taken.

19. Distributed Databases

Fragmentation and Commit

- A **distributed database** stores data across **multiple physically separated** sites that appear as **one** logical DB.
- The **3** data fragmentation types are **horizontal**, **vertical**, and **hybrid/mixed**.

- **Horizontal fragmentation** splits a relation by **rows (tuples)** using a **selection** predicate.
- **Vertical fragmentation** splits a relation by **columns** and must include the **primary key** in each fragment.
- **Data transparency** comes in **3** forms: **fragmentation**, **replication**, and **location** transparency.
- **Two-Phase Commit (2PC)** has **2** phases: **prepare/voting** and **commit/decision**, coordinated by a **coordinator**.
- **2PC** is a **blocking** protocol because participants wait if the **coordinator** fails.
- **Three-Phase Commit (3PC)** adds a **pre-commit** phase to make the protocol **non-blocking**.
- In **2PC**, a participant that voted **commit** cannot **unilaterally** change its decision.
- **Replication transparency** hides the existence of **multiple copies** from the user.

20. Query Processing and Optimization

Processing Stages

- Query processing has **3** steps: **parsing/translation**, **optimization**, and **evaluation**.
- A query is first translated into **relational algebra**, then into a **query evaluation plan**.
- The **2** cost-estimation approaches are **heuristic (rule-based)** and **cost-based** optimization.
- A **heuristic** rule pushes **selections and projections** as **early (down)** as possible in the tree.
- The **primary cost metric** for query plans is the number of **disk block (I/O) accesses**.
- The **2** join algorithms beyond nested-loop are **sort-merge join** and **hash join**.
- **Nested-loop join** costs **$O(m*n)$** block accesses in the naive case.
- **Block nested-loop join** reduces I/O by processing the inner relation **once per outer block**.
- **Materialization** stores **intermediate results**, while **pipelining** passes tuples **directly** to the next operator.
- **Selectivity** is the **fraction of rows** a predicate is expected to return.
- **Histograms** stored in the **catalog** help estimate **selectivity** for the optimizer.

21. Storage and Buffer Management

Disks, RAID, and Buffers

- The **storage hierarchy** from fastest to slowest is **registers, cache, main memory, flash, disk, tape**.
- **Primary storage** is **volatile** (RAM); **secondary** (disk) and **tertiary** (tape) are **non-volatile**.
- Disk access time has **3** components: **seek time**, **rotational latency**, and **transfer time**.
- **RAID 0** does **striping** for speed with **no redundancy** (no fault tolerance).
- **RAID 1** does **mirroring**, duplicating data with **50%** space overhead.
- **RAID 5** uses **block-level striping** with **distributed parity** and survives **1** disk failure.
- **RAID 6** adds **double parity** and survives **2** simultaneous disk failures.
- **RAID 10** combines **mirroring + striping** for both **speed and redundancy**.
- The **buffer manager** uses replacement policies like **LRU** and **clock** to evict pages.
- A **pinned** buffer page **cannot be evicted** while it is in use.

22. PL/SQL and Programmatic Access

Procedural Extensions

- **PL/SQL** is **Oracle's** procedural extension with a **block structure**: **DECLARE**, **BEGIN**, and **EXCEPTION**.

- A **cursor** is a **pointer to a result set**; the **2** types are **implicit** and **explicit**.
- The **4** explicit cursor operations are **DECLARE**, **OPEN**, **FETCH**, and **CLOSE**.
- The **4** common cursor attributes are **%FOUND**, **%NOTFOUND**, **%ROWCOUNT**, and **%ISOPEN**.
- **Embedded SQL** mixes SQL with a host language using the prefix **EXEC SQL**.
- **Impedance mismatch** is the gap between **set-oriented SQL** and **record-oriented** host languages.
- **Dynamic SQL** builds and executes statements at **runtime**, unlike **static** (compile-time) SQL.
- **JDBC** and **ODBC** are **4-step** APIs that let programs **connect**, **query**, and **process** results.
- A **stored function must return a value**, whereas a **stored procedure need not**.
- **Triggers** can be **row-level (FOR EACH ROW)** or **statement-level**.

23. SQL Deep Dive

Window Functions and Advanced Clauses

- **Window functions** compute across rows **without collapsing** them, using the **OVER()** clause.
- The **3** ranking window functions are **ROW_NUMBER()**, **RANK()**, and **DENSE_RANK()**.
- **RANK()** leaves **gaps** after ties, while **DENSE_RANK()** leaves **no gaps**.
- **PARTITION BY** divides rows into **groups** for a window function, like a **per-window GROUP BY**.
- **LAG()** and **LEAD()** access **previous** and **next** row values within a window.
- The **CTE (Common Table Expression)** uses the **WITH** keyword to define a **named temporary** result set.
- A **recursive CTE** references **itself** and is useful for **hierarchical/tree** queries.
- **LIMIT** (MySQL/PostgreSQL) and **TOP** (SQL Server) restrict the **number of returned rows**.
- **OFFSET-FETCH** provides **ANSI-standard** pagination of results.
- **MERGE** (a.k.a. **UPSERT**) performs **INSERT or UPDATE** in one statement based on a match.

Constraints and Keys Extra

- A **CHECK** constraint enforces a **boolean condition** on each row's column values.
- A **composite primary key** spans **2 or more** columns and must be **collectively unique**.
- A **surrogate key** is a **system-generated** key (e.g., **IDENTITY/AUTO_INCREMENT/SEQUENCE**) with **no business meaning**.
- A **natural key** is derived from **real-world data** (e.g., **email** or **SSN**).
- **Deferred constraints** are checked at **transaction commit** instead of per-statement.
- A **self-referencing foreign key** points to the **same table** (e.g., **employee -> manager**).

24. NoSQL Expanded

Key-Value Stores

- **Redis** is an **in-memory** key-value store supporting **5** core data types: **strings**, **lists**, **sets**, **sorted sets**, and **hashes**.
- **Redis** provides **persistence** via **2** mechanisms: **RDB snapshots** and **AOF (append-only file)**.
- **Redis** is commonly used for **caching**, **session storage**, and **leaderboards** (sorted sets).
- **Memcached** is a **volatile**, **multi-threaded** cache with **no persistence** and **no data types**.
- **DynamoDB** uses a **partition key** (and optional **sort key**) to locate items.
- Key-value stores excel at **O(1)** lookups but **cannot query** by the value's **internal fields**.
- **Riak** and **Voldemort** are key-value stores influenced by the **Amazon Dynamo** paper.

- Redis **TTL (time-to-live)** enables **automatic key expiry** for cache entries.

Document Stores

- **MongoDB** groups documents into **collections**, the analog of relational **tables**.
- A **MongoDB document** is stored as **BSON** (Binary JSON) with a mandatory **_id** primary field.
- **MongoDB** replication uses a **replica set** with **1 primary** and multiple **secondaries**.
- **MongoDB** elects a new **primary** automatically via a **majority vote** on primary failure.
- The **MongoDB aggregation pipeline** chains stages like **\$match**, **\$group**, and **\$sort**.
- **MongoDB** sharding distributes a collection across shards using a **shard key**.
- **CouchDB** exposes a **RESTful HTTP/JSON** API and uses **MVCC** for conflict handling.
- Document stores allow **schema flexibility**, so documents in one collection **may differ** in fields.
- **Embedding** stores related data **in one document**; **referencing** links via **_id** like a foreign key.

Column-Family Stores

- **Cassandra** originated at **Facebook** and combines **Dynamo's** distribution with **Bigtable's** data model.
- **Cassandra** has **no single master** (peer-to-peer), avoiding a **single point of failure**.
- **Cassandra** tunable consistency uses levels like **ONE**, **QUORUM**, and **ALL** per query.
- **HBase** runs on **HDFS** and is the **open-source** implementation of **Google Bigtable**.
- A **column family** groups **related columns** stored together on disk.
- **Cassandra** writes go first to a **commit log** and an in-memory **memtable**, later flushed to an **SSTable**.
- **Wide-column** stores are optimized for **high write throughput** and **time-series** data.

Graph Stores

- **Neo4j** stores **nodes**, **relationships**, and **properties**, querying with **Cypher**.
- Graph traversals avoid the costly **joins** of relational models for **connected** data.
- The **property graph** model attaches **key-value properties** to both **nodes and edges**.
- **Graph** databases power **social networks**, **recommendation engines**, and **fraud detection**.
- An **RDF triple store** represents data as **subject-predicate-object** statements queried by **SPARQL**.

CAP, PACELC, and Consistency

- **CAP** forces a choice of **2 of 3: Consistency, Availability, Partition tolerance**.
- The **PACELC theorem** extends CAP: if **Partition** then **A vs C, Else Latency vs Consistency**.
- **Read-your-writes** consistency guarantees a user sees their **own latest update**.
- **Monotonic read** consistency guarantees reads **never go backward** in time.
- **Causal consistency** preserves the order of **causally related** operations.
- A **quorum** requires **$R + W > N$** for strong consistency and **$W > N/2$** to avoid **write conflicts**.
- **Vector clocks** detect **concurrent updates** and **conflicts** in distributed stores like **Dynamo**.
- **Conflict resolution** strategies include **last-write-wins (LWW)** and **application-level merge**.
- **Eventual consistency** guarantees convergence **if no new writes** occur for a while.

Sharding and Replication Extra

- **Range-based** sharding keeps **ordered** data together, aiding **range scans** but risking **hotspots**.
- **Hash-based** sharding distributes **evenly** but loses **range-query** locality.
- **Consistent hashing** maps both **keys and nodes** onto a **ring**, moving only **K/N** keys on change.

- **Virtual nodes (vnodes)** improve **load balancing** in consistent-hashing rings.
- **Leaderless replication** (e.g., **Dynamo**, **Cassandra**) lets **any replica** accept writes.
- **Read repair** fixes **stale replicas** during a read in eventually consistent systems.
- **Anti-entropy** background processes (e.g., **Merkle trees**) reconcile **replica differences**.
- **Write amplification** occurs when one logical write triggers **many physical** writes (e.g., **LSM-tree** compaction).

NoSQL vs SQL Comparison

- **SQL** databases use a **fixed schema**; **NoSQL** databases are typically **schema-less/flexible**.
- **SQL** guarantees **ACID**; many **NoSQL** stores favor **BASE** and **eventual consistency**.
- **SQL** scales **vertically**; **NoSQL** is designed to scale **horizontally**.
- **NewSQL** systems (e.g., **Google Spanner**, **CockroachDB**) aim for **ACID + horizontal scale**.
- **Polyglot persistence** means using **different databases** for **different needs** in one application.
- **LSM-trees** (used by Cassandra/HBase) optimize **writes**, while **B+ trees** (RDBMS) balance **read/write**.

25. More Normalization

Higher Normal Forms

- A **multivalued dependency (MVD)** $X \twoheadrightarrow Y$ exists when **Y** is independent of the **rest** given **X**.
- **4NF** requires that every non-trivial **MVD** has a **super key** on its left side.
- **5NF (PJ/NF)** eliminates **join dependencies** not implied by **candidate keys**.
- **DKNF (Domain-Key Normal Form)** requires all constraints to follow from **domain and key** constraints.
- A **lossless decomposition** into **BCNF** always exists, but it may sacrifice **dependency preservation**.
- **Denormalization** intentionally adds **redundancy** to improve **read performance** at the cost of **write anomalies**.
- **3NF synthesis** algorithm builds relations from the **minimal cover** guaranteeing **lossless + dependency-preserving** results.

FD Computation Extra

- An **extraneous attribute** in an FD can be **removed** without changing the **closure** of the FD set.
- Two attribute sets have the **same closure** iff each **determines** the other.
- A relation with **n** attributes can have at most **2ⁿ** distinct attribute subsets to test for keys.
- If every attribute is **prime**, the relation is automatically in **3NF**.
- A **partial dependency** cannot exist if the **primary key** is a **single attribute**.

26. More ER and Relational Modeling

Additional Modeling Concepts

- A **ternary** relationship cannot always be losslessly replaced by **3 binary** relationships.
- The **(min, max)** notation specifies **participation** and **cardinality** together on each edge.
- A **role name** disambiguates the **2** ends of a **recursive** relationship.
- **Cardinality of a relationship** can carry **descriptive attributes** drawn on the **diamond**.
- An **entity** converting to a **relationship** (or vice versa) reflects a **design choice**, not a fixed rule.
- A **ternary relationship** maps to a relation whose key is the **combination of all 3** participating keys.

Relational Calculus Extra

- **TRC** uses the form $\{t \mid P(t)\}$, where **t** is a **tuple variable**.
- **DRC** uses the form $\{ \langle x_1, \dots, x_n \rangle \mid P(x_1, \dots, x_n) \}$ over **domain** variables.
- An **unsafe** expression can produce an **infinite** result and is **disallowed**.
- The **universal quantifier** ($\forall$) can be rewritten using $\neg \exists \neg$ (negated existential).
- **Relational completeness** means a language is at least as expressive as **relational algebra**.

27. Object and Temporal Concepts

Beyond Relational

- An **ORDBMS** (e.g., **PostgreSQL**) extends the relational model with **user-defined types** and **inheritance**.
- **OODBMS** stores **objects directly**, eliminating **impedance mismatch** with OOP languages.
- A **temporal database** tracks **2** time dimensions: **valid time** and **transaction time**.
- A **spatial database** stores **geometric/geographic** data using indexes like the **R-tree**.
- **BLOB** stores **binary large objects** (images), while **CLOB** stores **character large objects** (text).

28. Data Warehousing and OLAP

Analytical Processing

- **OLTP** handles **many short transactions** (writes), while **OLAP** handles **complex analytical** reads.
- A **data warehouse** is **subject-oriented**, **integrated**, **time-variant**, and **non-volatile** (Inmon's **4** traits).
- The **star schema** has **1** central **fact table** surrounded by **denormalized dimension** tables.
- The **snowflake schema normalizes** dimension tables into multiple related tables.
- **ETL** stands for **Extract**, **Transform**, and **Load**.
- The **3** OLAP operations are **roll-up (aggregate)**, **drill-down (detail)**, and **slice/dice**.
- A **data mart** is a **subject-specific subset** of a data warehouse.
- **Bitmap indexes** suit **low-cardinality** warehouse columns; **B+ trees** suit **high-cardinality** OLTP columns.

29. More Indexing and Hashing

Hash and Tree Structures

- **ISAM** is a **static** indexed-sequential structure that degrades with **overflow chains**.
- A **hash function** maps a **search key** to a **bucket address**.
- **Collision** occurs when **2** keys hash to the **same bucket**.
- The **2** static-hashing collision handlers are **open addressing** and **separate chaining (overflow buckets)**.
- **Dynamic hashing** adapts the **bucket count** as the file **grows/shrinks**.
- **Linear hashing** grows buckets **one at a time** without a directory.
- **Extendible hashing** uses **global depth** for the directory and **local depth** per bucket.
- A **B-tree** of order m holds keys between $\text{ceil}(m/2)-1$ and $m-1$ per node.
- **Bitmap index** uses one **bit per row per value**, enabling fast **AND/OR** on multiple columns.
- **Covering index** answers a query **entirely from the index** without touching the **table**.

30. Extra Exam Recall

High-Yield Numeric and Naming Facts

- There are **4** SQL isolation levels: **Read Uncommitted**, **Read Committed**, **Repeatable Read**, and **Serializable**.
- There are **3** ARIES recovery phases: **Analysis**, **Redo**, and **Undo**.
- There are **3** data fragmentation types: **horizontal**, **vertical**, and **mixed**.
- There are **2** phases in **2PC**: **voting/prepare** and **commit/decision**.
- There are **3** intention-lock types: **IS**, **IX**, and **SIX**.
- There are **5** Redis core data types: **strings**, **lists**, **sets**, **sorted sets**, and **hashes**.
- There are **3** ranking window functions: **ROW_NUMBER()**, **RANK()**, and **DENSE_RANK()**.
- There are **3** OLAP operations: **roll-up**, **drill-down**, and **slice/dice**.
- **Inmon** lists **4** warehouse traits: **subject-oriented**, **integrated**, **time-variant**, and **non-volatile**.
- **RAID 5** survives **1** disk failure; **RAID 6** survives **2**.
- The **quorum** rule for strong consistency is **$R + W > N$** .
- **ETL** = **Extract, Transform, Load**.
- **HBase** is **CP**; **Cassandra** is **AP** under the CAP theorem.
- **MongoDB** default port is **27017**; **Cassandra** CQL is **9042**; **Redis** is **6379**; **Neo4j Bolt** is **7687**.
- **PostgreSQL** default port is **5432**; **MySQL** is **3306**; **Oracle** is **1521**; **SQL Server** is **1433**.
- **MongoDB** replica sets elect a primary by **majority vote**.
- **Disk access time** = **seek time + rotational latency + transfer time**.
- The cheapest relational-algebra optimization is to **push selections down** the tree first.
- A **covering index** avoids accessing the **base table** entirely.
- **Snapshot isolation** can still permit the **write-skew** anomaly.